



4EYES Labs Audit

Dexy Gold Stablecoin Protocol

Auditor: Luca D'Angelo

Date: 2025-01-12

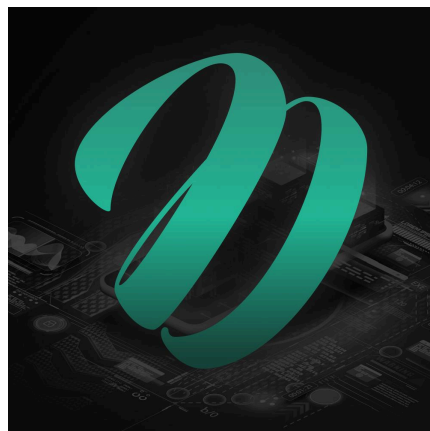


Table of Contents

Disclaimer	3
Audit Summary	4
Contracts	5
Bank Contract	5
Arbitrage Mint Contract	6
Free Mint Contract	8
Intervention Contract	10
Payout Contract	12
BuyBack Contract	14
Update Contract	16
Ballot Contract	18
Main LP Pool Contract	19
LP Redeem Contract	20
LP Extract for the Future Contract	22
LP Mint Contract	26
LP Swap Contract	27
Tracking Contract	28
Deposit Proxy Contract	30
Redeem Proxy Contract	32
SwapBuy Proxy Contract	34
SwapSell Proxy Contract	36
Appendix	38
Issue Guide	38
About Us	38

Disclaimer

This audit is provided as-is at the time of its completion and might not reflect the most up-to-date state of any smart-contracts deployed, or any design document, or any research paper referenced herein.

Audit Summary

Protocol	Dexy Gold Stablecoin Protocol
Commit	c12466cdef85ffc4151a4e7c3b38fa1c713c08d0
Repository	https://github.com/kushti/dexy-stable

Contract	File Name	Severity	Status
Bank	bank.es	GOOD	RESOLVED
Arbitrage Mint	arbmint.es	WARNING	RESOLVED
Free Mint	freemint.es	SERIOUS	RESOLVED
Intervention	intervention.es	CRITICAL	RESOLVED
Payout	payout.es	SERIOUS	RESOLVED
BuyBack	buyback.es	GOOD	RESOLVED
Update	update.es	WARNING	RESOLVED
Ballot	ballot.es	GOOD	RESOLVED
LP Main	main.es	GOOD	RESOLVED
LP Redeem	redeem.es	WARNING	RESOLVED
LP Extract for the Future	extract.es	WARNING	RESOLVED
LP Mint	mint.es	GOOD	RESOLVED
LP Swap	swap.es	SERIOUS	RESOLVED
Tracking	tracking.es	WARNING	RESOLVED
Deposit Proxy	Deposit.es	GOOD	RESOLVED
Redeem Proxy	Redeem.es	GOOD	RESOLVED
SwapBuy Proxy	SwapBuyV2.es	WARNING	RESOLVED
SwapSell Proxy	SwapSell.es	WARNING	RESOLVED

Contracts

Bank Contract

- Contains bank singleton NFT identifier.
- Contains dexyGold stablecoin tokens to emit.
- Involved in the following transaction:
 - Arbitrage Mint
 - Free Mint
 - Intervention
 - Payout
 - Update
- Logic of each transaction is separated in a specific contract instead of all inside the bank box.
- Bank contract just checks that valid boxes are included in different transactions based on their singleton NFT identifier.

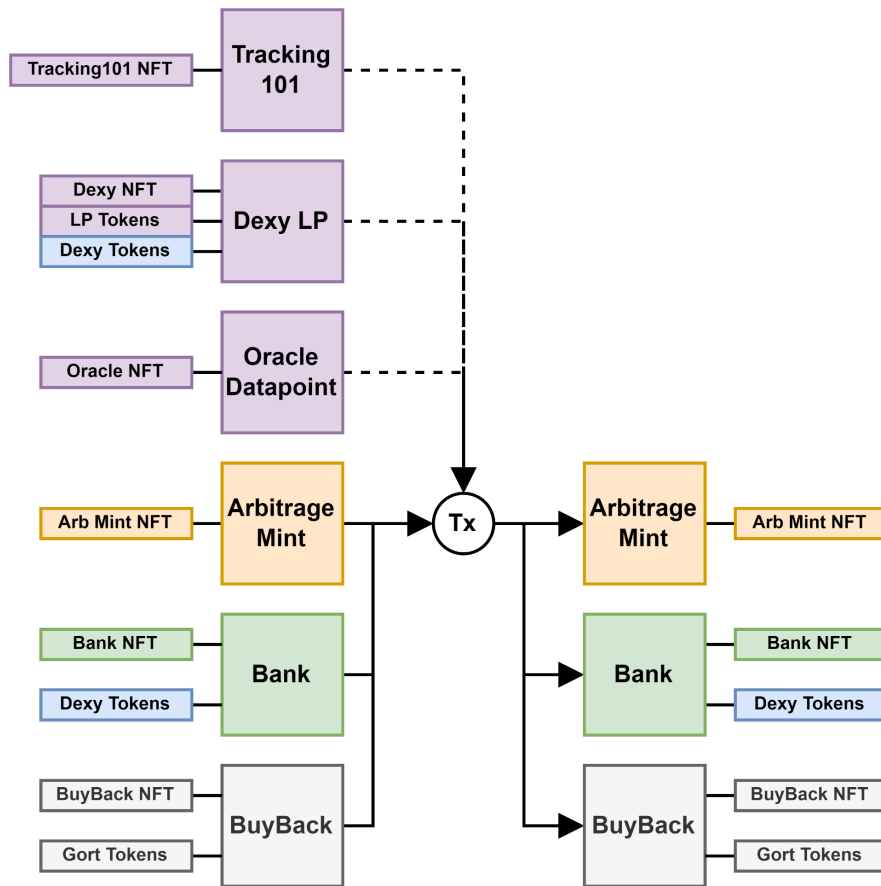
Arbitrage Mint Contract

- Contains arbitrage mint singleton NFT identifier.
- R4 contains the block height at which the T_arb period counter will reset.
- R5 contains the remaining dexyUSD tokens available to be purchased before the counter resets.
- L46-L48 defines
 - T_arb period for arbitrage minting.
 - T_buffer max delay for changing state of this box when waiting for tx confirmation.
 - thresholdPercent which means when $r \geq 1.01$ for an amount of time exceeding T_arb, arbitrage minting is permitted.
- L50-L52 defines the arbitrage mint fee, 0.3% for the bank and 0.2% for the buyback fees, so a cumulative fee of 0.5%, as outlined in the whitepaper.
- L75 defines isCounterReset condition for knowing when the T_arb period counter has been reset.
- L80 defines oracleRate which is the price of gold for Dexy protocol in units of nanoErg per 1mg of Gold, but the oracle gives nanoErg per 1kg of Gold, so must divide this value by 1000000.
- L82-L84 defines lpRate which is the price of dexy in the liquidity pool.
- L86-L89 checks that the correct amount of ERG was deposited in the bank reserve based on the oracle price and the bank fee.
- L91-L93 checks that the oracle buyback box gets the correct amount of ERG based on the oracle price and the buyback box.
- L95 defines oracleRateWithFee as the bankRate added to the buybackRate. The oracle rate means the buyback rate and the fee means the bank fee. This defines the total fee paid in ERG by the user in order to purchase Dexy tokens.
- L99 defines maxAllowedIfReset which corresponds to δ_u in the whitepaper.
 - In the paper, the price is defined as the inverse of the price in the smart-contract, i.e. the paper defines the price in units of "dexy per erg" while contract is defined as "erg per dexy" (really nanoErg per 1 mg of gold), thus the inequalities in the paper are flipped in the contract.
 - Considering the flipped inequalities, we could define a similar equation as in the paper involving the square root of the product of the oracle price, amount of stablecoins, and ERG in the LP. The smart-contract uses the simplified equation without the square root, however, the equation with the square root and the simplified equation actually represent two *different* values, so it would be best to reflect that in the paper somehow.
 - The δ_u with the square root is a smaller value than the simplified equation without the square root.
 - L109 defines the quantity available to mint based on isCounterReset. If the counter is not reset, the amount left for minting is defined in R5 of the arbitrage mint box.

4EYES Labs

- L111 checks that the amount of dexy minted, represented by dexyMinted, is less than or equal to what is available left to mint, represented by availableToMint.
- L113-L118 defines how the height at which the counter will be reset. If the counter has not been reset, then we propagate the value to the successor box. If the counter has been reset, we set the next counter reset height to the current height + the T_arb period + the T_buffer period.
- L120 updates R5 to be the remaining amount of stablecoins that can be minted during the arbitrage mint phase, which is just the difference of availableToMint and dexyMinted.
- L122-L131 just checks that correct boxes are used in inputs by validating their singleton NFT identifiers, as well as making sure the arbitrage box is recreated properly.
- L133-L134 checks that the tracking height is correct, which ensures that the arbitrage mint only occurs after T_arb amount of time has elapsed while the liquidity pool price is greater than 1.01 times the oracle price, i.e. oracle price is less than liquidity pool price by 1 percent.

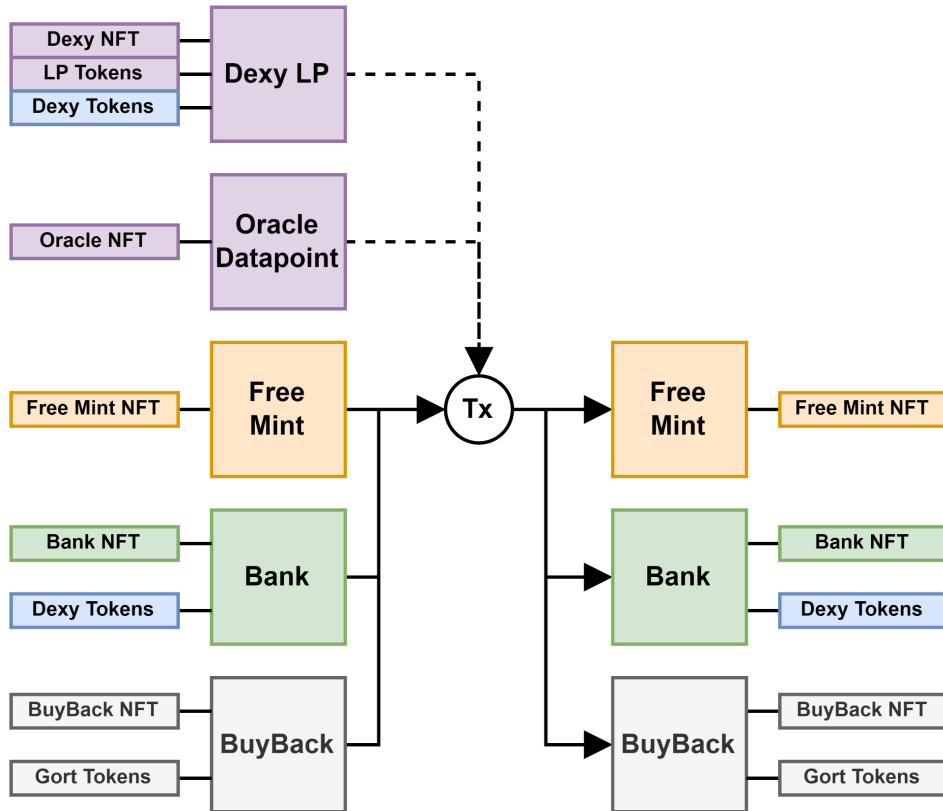
Transaction Diagram



Free Mint Contract

- Contains free mint singleton NFT identifier.
- R4 is the blockheight at which the free mint period counter will reset.
- R5 contains remaining Dexy stablecoins available for minting during the free mint period and before the counter is reset.
- L44 defines the free mint period T_{free} as 360 blocks, or roughly half of one day.
- L45 defines the buffer period T_{buffer} for the allowed delay for modifying this box.
- L47-L49 defines the free minting fee, with 0.3% for the bank fee and 0.2% for the buyback fee, so a cumulative fee of 0.5% as outlined in the whitepaper.
- L69 defines isCounterReset condition to check if the current HEIGHT is greater than the previous counter reset height.
- L74 defines the oracleRate in nanoErg per 1mg of Gold.
- L78 defines the lpRate from the liquidity pool box data-input.
- L80 defines the validRateFreeMint as the lpRate being greater than 98% the oracleRate. However, it does not provide an upper-bound for the free mint as in the whitepaper.
- L82-L85 defines how much stablecoins have been minted by the user, the amount of ERG added into the bank, the bankRate, and checks that the correct amount of ERG was added into the bank based on the amount of Dexy minted and the bank rate.
- L87-L90 checks that the oracle buyback box get the correct amount of ERG based on the buyBackRate and dexyMinted.
- L92 defines maxAllowedIfReset as 1% of the stablecoin amount in the liquidity pool, as in the whitepaper.
- L94 defines availableToMint as the valid amount of stablecoins that can be minted during the free mint, depending on if the T_{free} period counter has been reset or not.
- L96 defines validAmount and checks if the dexyMinted amount is less than the availableToMint amount.
- L98-L103 propagates the counter reset height or assigns a new one if the counter has been reset.
- L104 checks that the remaining amount of Dexy stablecoins that is available to mint is updated correctly.
- L106-L118 just checks that correct boxes are used in inputs by validating their singleton NFT identifiers, as well as making sure the free mint box is recreated properly.

Transaction Diagram

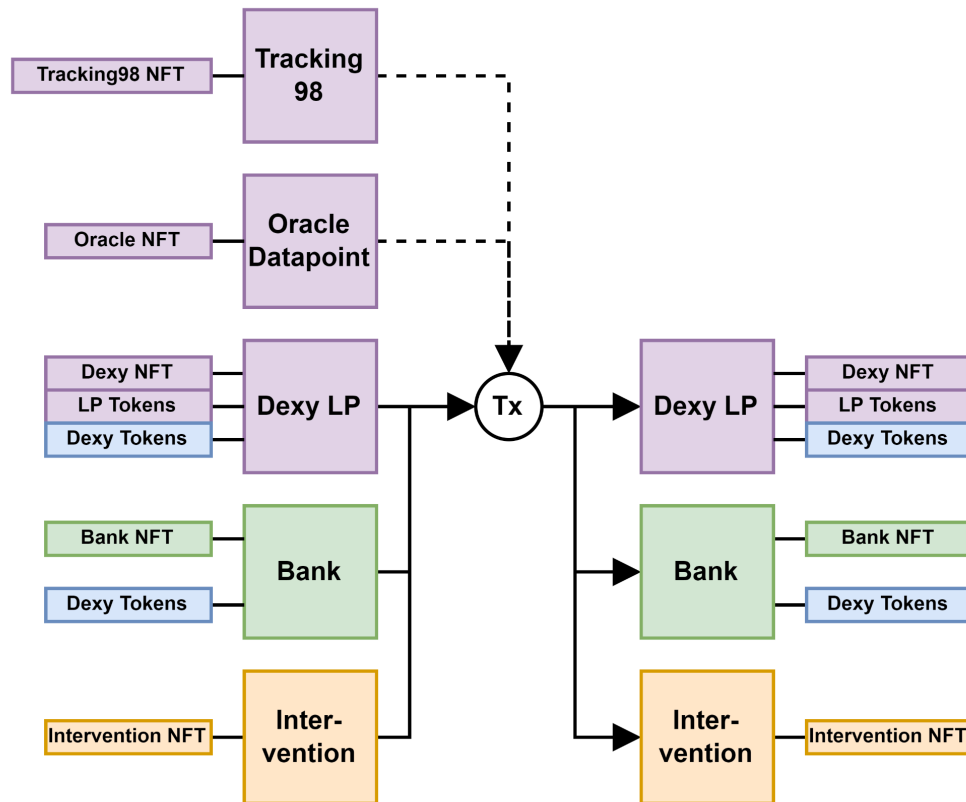


Intervention Contract

- Contains the intervention singleton NFT identifier.
- L39 defines lastIntervention which is the creation height of the current input intervention box, representing the previous intervention block height.
- L40 defines the buffer period allowed for the intervention block height.
- L41 defines the intervention period T as 360 blocks representing roughly 1 day, corresponding to T_int in the whitepaper.
- L42 defines T_int as 20, representing the amount of blocks after which a trigger swap event can happen, as long as the liquidity pool rate is still less than the oracle pool rate.
 - This concept is not really explained in the Intervention section of the whitepaper, it should be included since it is not clear what this is for in the contract.
- L49 defines thresholdPercent, which means the intervention will happen when the liquidity pool price is less than or equal to 98% of the oracle price, as in the whitepaper, but inverted due to how the price is defined in the contracts.
- L54 checks what type of transaction the intervention box is involved with, if no update transaction, then normal intervention logic continues.
- L72-L76 gets the amount of ERG and Dexy stablecoin tokens from the liquidity pool input and output boxes.
- L84 defines the oracleRateXy as nanoErg per 1mg of Gold.
- L86 defines validThreshold checking that the liquidity pool rate is at most 98% the oracle rate.
- L97-L99 checks that the intervention box is recreated correctly, making sure that the creation height of the successor accounts for the buffer period.
- L102 checks that at least an amount of time equivalent to the intervention period T has elapsed since the lastIntervention block height, as in the white paper.
- L104-L107 defines the difference in amounts for ERG and Dexy stablecoins in the bank and liquidity pool input and output boxes.
- L109 defines trackingHeight from the tracking box for the current intervention event.
- L111 checks that at least T_int blocks have passed since the start of tracking at trackingHeight.
- L113 checks that:
 - The liquidity pool rate is not more than the 99.5% the oracle rate, *after* the intervention.
 - It also checks that the bank spends no more than 1% of its reserves for the intervention.
 - This is contrary to the 2% from the whitepaper.
- L117 defines price as the input liquidity pool box price for the Dexy stablecoin token.
- L119 checks that:
 - Reserves used by the bank are at most what was added to the liquidity pool.
 - Bank accepts greater than or equal to the amount of Dexy tokens removed from the liquidity pool.

- That the constant product rule is respected, represented as an inequality since the amount of Dexy removed from the liquidity pool and the amount of ERG added to the liquidity may not all come from the bank but a swap as well.
- The ERG added to the liquidity pool is greater than 0.
- The ERG removed from the bank reserves represents less than or equal to 2% more than the corresponding value of Dexy tokens removed from the liquidity pool.

Transaction Diagram



Payout Contract

- Contract which enforces the logic for how rewards should be distributed to the buyback contract, which will be used to pay oracle operators.
- R4 contains an integer representing the height when the last payment occurred, being initially set to 0.
- L20 comment says the hash of the script should be stored in R4 of the payout box but this is contrary to what is actually stored in R4 of the payout box, which stores the height of the last payment instead.
- L22-L24 defines the payoutThreshold at 100000 ERG, maxPayOut at 100 ERG, and minPayOut at 10 ERG.
- L52 defines validOracle checking that the oracle box contains the correct oracleNFT.
- L54 gets the height for the output payout box but gets a Coll[Byte] instead Int type from R4. (this variable is actually never used)
- L56 defines bankDexy as the amount of Dexy tokens in the input bank box reserve.
- L61 defines oracleRate as the amount of nanoErgs per 1mg of gold.
- L63 defines dexyInCirculation as the difference between a compile time constant called \$initialDexyTokens and bankDexy.
- L64 defines a boolean called collateralized that checks if the ERG in the bank reserve is greater than 500% collateralization level when compared to the dexyInCirculation.
 - This equation is not defined correctly, since the oracleRate is in units of nanoErg per 1mg of gold, it should be defined as:

```
// The amount of ERG in the reserves should be greater than
// 5 times the equivalent amount of ERG corresponding
// to the amount of Dexy in circulation.
val collateralized = oracleRate * dexyInCirculation * 5 < bankBoxIn.value
```

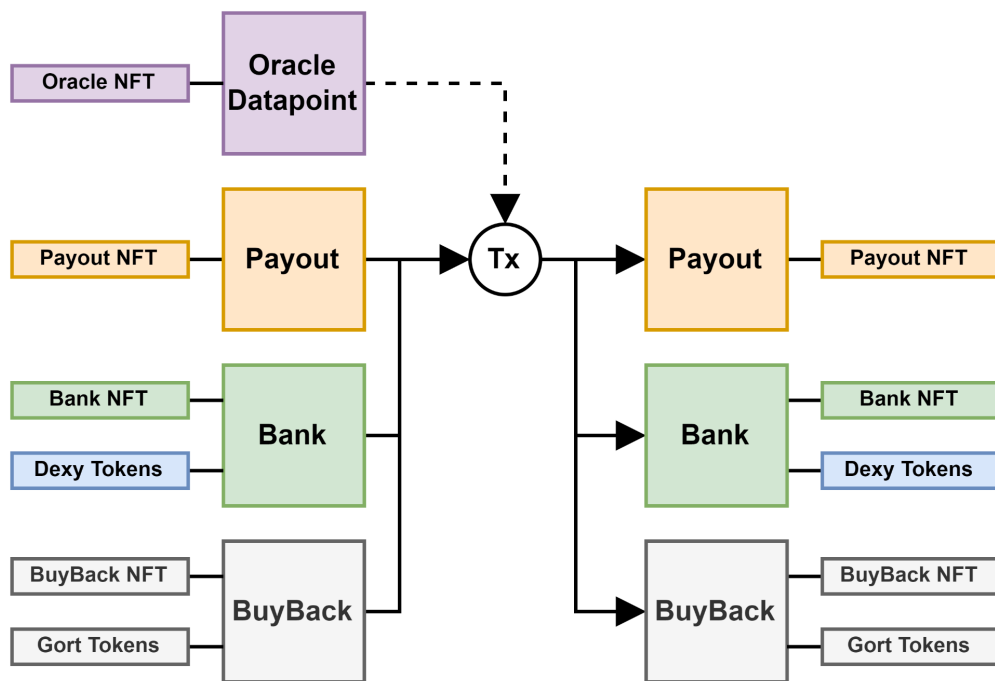
- L66 defines paymentAmount as a maximum of 0.5 % of the bank ERG reserves.
- L67 checks for properPayment condition that is satisfied if:
 - The difference in ERG from the input and output bank reserve strictly equals the paymentAmount.
 - The difference in ERG from the output and input buyback boxes strictly equals the paymentAmount, hence the paymentAmount from the bank goes to the buyback output box.
 - Typo in L68 that should instead be written as:

```
buybackBoxOut.value not buybackBoxOut. value
```

- L69 defines lastPayment as the integer in R4 of the current input payout box.
- L70 defines the variable buffer as the error margin in the payment height, set to 5 blocks.
- L71 defines delayInPayments, set to 5040 blocks, roughly one week.

- L72 checks for properHeight condition ensuring that the sum of lastPayment height and delayInPayments is less than or equal to the current height.
- L73 checks for properNewR4 condition ensuring that the output payout box new previous payment height is greater than or equal to the current height minus the buffer. The buffer is used in case there is delay whenever the transaction is confirmed vs the height when the transaction was built.
- L77 checks validBank to ensure nothing changes about the bank tokens.
- L79 checks validSuccessor to ensure that the output payout box is recreated correctly, notably the output payout box can have increased ERG value.
- L84 defines validBuyBackIn checking that the input buyback box contains the correct buybackNft.
- L86 defines validPayout checking that validBuyBackIn, collateralized, properPayment, and properHeight are correct.

Transaction Diagram



BuyBack Contract

- Accepts ERg via top-up action, then buys back GORT from the LP pool, then distributes GORT back to the oracle pool to reward oracle operators.
- Contains two tokens, a buyback NFT and at least 1 GORT token.
- Involved in three transactions: buyback, top-up, and give-back.
- L37 defines action as an integer, which is a context variable at index representing the transaction to execute in the control flow branches.
- (action = 0) for buyback from GORT LP pool.
 - L46 checks the poolInput contains the correct \$gortLpNFT
 - L51 defines a function called noTokens: (Box => Boolean) checking if a box has no tokens inside of it.
 - L52 checks that all outputs starting from index 2 contain no tokens. The first two output boxes are the LP and buyback boxes, which are the only ones that should contain GORT.
 - L55 defines selfOutCorrect checking that the output buyback box contains the buybackNft and the second token has as id the \$gortId since it should contain the GORTs it "bought back" from the liquidity pool.
 - L58 defines price as the ratio of the poolInput ERG value divided by the poolInput GORT token amount, so the units are ERG per GORT.
 - L59 defines gortObtained as the positive difference between the amount of GORT tokens in the output buyback box vs the amount of GORT tokens in the input (SELF) buyback box. This value should have increased.
 - L60 defines maxErgDelta as a slippage percentage equal to 5% more than the equivalent amount of gortObtained in ERG.
 - L61 defines ergDelta as the difference in ERG value of the input buyback box and the output buyback box. This represents the amount of ERG used to purchase the GORT from the LP pool.
 - L63 defines swapCorrect checking that the gortObtained is greater than 0, that ergDelta is less than or equal to maxErgDelta, and that the ERG added into the LP pool is equal to ergDelta.
- (action = 1) for top-up adding ERG into the box without modifying anything else,
 - L70 defines topUp checking that the output buyback box tokens are the same as the input buyback box tokens, and that the proposition bytes are equal, as well as ensuring that the output buyback box ERG value has increased and is greater than the input buyback box value.
- (else) for giving back GORT to the oracle pool, which will then use them to reward oracle operators.
 - L82-L95 we assume it is copied correctly from the oracle pool contract.
 - L97 defines selfGort as the amount of GORT tokens inside the input buyback box minus 1.
 - L98 defines properGiving checking that the input oracle pool box poolIn contains the correct \$oracleNFT and that the output oracle pool box contains greater or

4EYES Labs

equal to the amount of GORT tokens in the input oracle pool box poolIn plus the amount selfGort minus the rewardEmitted amount.

- L101 defines giveback checking that the output buyback box contains the same buyback NFT as the input, that it contains the same GORT token id, that the contract is the same, that the ERG value in the box is the same, and that properGiving is satisfied.

Update Contract

- L13 defines `contractToUpdateNFT`, which will take on the value of the singleton token id corresponding to either the `bankNFT`, `extractNFT`, or `interventionNFT`.
 - This means that there is an update contract compiled for each of these three contracts.
- L15 defines `ballotTokenId`, which will be the token id used for ballots when users cast their votes.
- L18 defines `minVotes` as 3, which will be the minimum threshold to reach in order to update a contract.
- L21 defines `updateBoxOut` as the first output, so the current contract will be recreated as the first output of the transaction.
- L23 defines `validUpdateIn`, checking that the current update contract box input is the first input in the transaction.
- L25 defines `contractToUpdateBoxIn` as the second input in the transaction, so this will be the box getting updated to a new contract.
- L26 defines `contractToUpdateBoxOut` as the second output in the transaction, so the contract being updated will be recreated with a new contract as the second output of the transaction.
- L29 defines `contractToUpdateBoxOutHash` as the blake2b256 hash of the box proposition bytes (i.e. `ergotree` bytes) of the `contractToUpdateBoxOut` output box.
 - So this is just the hash of the new contract, the update of the previous contract for the `contractToUpdateBoxIn`.
 - This will be the value that users will be voting on.
- L31 defines `validContractToUpdateIn` that checks if the `contractToUpdateIn` has greater than or equal to one token inside of it and that the first token has the correct `contractToUpdateNFT` token id
- L34 defines `validContractToUpdateOut` that checks if the tokens inside the `contractToUpdateBoxIn` and `contractToUpdateBoxOut` are the equal and their box values are also equal.
- L39 defines `validUpdateOut` that checks if the current input update contract box has the same tokens as the `updateBoxOut`, and that the update contract remains the same, as well as the `updateBoxOut` value being greater or equal to the current input update contract box value.
- L43 defines `isValidBallot` as a function accepting a `Box` and checking that it contains more than zero tokens, and that the first token has the `ballotTokenId` as its id, and that `R5` contains a collection of bytes corresponding to the id of the current input update box (i.e. thus votes/ballots correspond to a particular update box), and that `R6` contains a collection of bytes corresponding to `contractToUpdateBoxOutHash`, which is the value being voted on.
- L50 defines `ballotBoxes` as the set of input boxes corresponding to valid ballot boxes, using the function `isValidBallot` as a filter.
- L52 defines `votesCount` as the sum of the amount of ballot tokens inside each ballot box.

4EYES Labs

- How does voting work exactly? Does each ballot box contain only one ballot token? If so, then would this just not be the amount of valid ballot boxes satisfying the filter?
- L54 defines validVotes checking that votesCount is greater than or equal to minVotes.

Ballot Contract

- R4 contains the GroupElement of the owner of the ballot box.
- R5 contains the id of the update box as Coll[Byte] type. Each new vote to update a contract to a new contract involves a different update box.
- R6 contains the hash of the proposition bytes of the new contract that is replacing the old contract, as Coll[Byte] type.
- L6 defines updateNFT as the token id held in the update box. Since it is a compile time constant, this means a new ballot contract is compiled whenever a new vote (i.e. voting session) happens.
- L8 defines pubKey as the GroupElement of the user owning the ballot in R4 of the current ballot box (i.e. SELF).
- L10 defines index as the index where the current ballot box exists in the inputs of the transaction.
- L12 defines output as the recreated ballot box in the outputs of the transaction, which should be in the same location as the current input ballot box.
- L14 defines isBasicCopy checking that the output ballot box has the same GroupElement in R4 as pubKey, and that the output proposition bytes are the same as the current ballot box, and that the output tokens are the same, and that the output ballot box value is greater than or equal to the current input ballot box.
- L19 defines castVote as the SigmaProp corresponding to the discrete logarithm of pubKey.
- L21 defines notVoted as checking if R5 of the output ballot box is not defined. Thus an output ballot box cannot contain the box id of the update box, thus potentially counting as a second vote.
- L22 defines update as checking if the first input to the transaction is the update box containing the updateNFT and that notVoted evaluates to true.
- L31 checks that the box can be spent only if isBasicCopy is true and that either castVote is true or update is true.
 - If update is false, then the owner can spend this box by recreating it.
 - If castVote is false, then an update must occur, which does not require the owner to sign their input ballot box for the transaction.

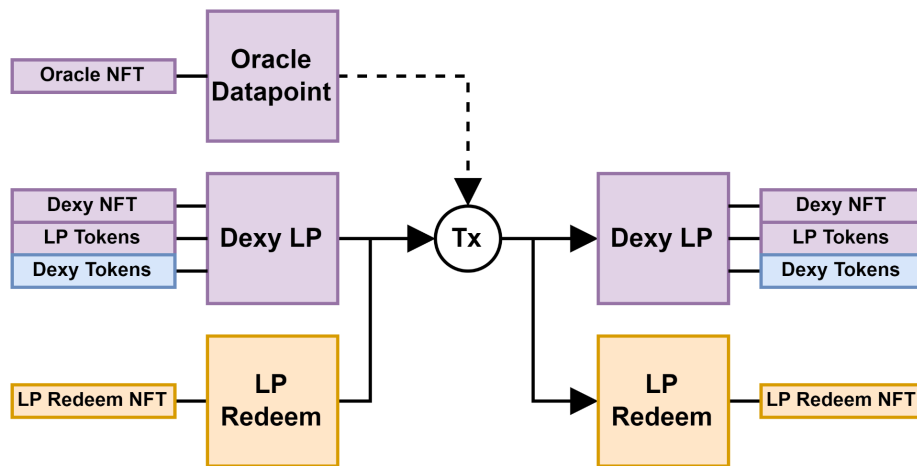
Main LP Pool Contract

- The LP box contains the following tokens:
 - NFT singleton
 - LP tokens
 - Dexy tokens
- The dexy Gold tokens are the Y tokens, the nanoErg in the box are the X tokens.
- There are 6 transactions where the LP pool is involved:
 - Intervention
 - Swap
 - Redeem LP tokens
 - Mint LP tokens
 - Extract to future
 - Release extracted to future tokens
- L74-L78 defines the singleton NFT token ids for the different action boxes.
- L88-L93 checks that an action path is valid if the corresponding action box contains the correct singleton NFT.
- L103-L106 checks that the LP box contract, the LP singleton NFT id, the LP tokens id, and the dexy tokens id remain the same when the output LP box is recreated.
- L116 defines deltaSupplyLp as the difference between the amount of LP tokens in the input LP box and the amount of LP tokens in the output LP box.
- L119 defines noMoreTokens checking that the output LP box contains only 3 different token types: LP singleton, LP tokens, Dexy tokens.
- L121 defines lpAction checking which action path is being taken, either swap, mint, or redeem.
- L123 defines dexyAction checking if there is a dexy bank action, either an intervention or an extraction, while also making sure that no LP tokens are added or removed from the LP box.

LP Redeem Contract

- Contract used for redeeming LP tokens from the liquidity pool.
 - Though not explicitly explained in the whitepaper as “redeeming LP tokens from the liquidity pool” this action corresponds to the action of “stopped withdrawals” wherein LP providers cannot withdraw their ERG from the pool if $r \leq 0.98$.
- Contains singleton NFT for identifying this box.
- L9 defines initialLp as a compile time constant defining the initial amount of LP tokens minted.
- L35 defines supplyLpIn as the initialLp minus the amount of lpReservesIn tokens. This variable represents the circulating supply of LP tokens with respect to the inputs of the transaction.
- L40 defines oracleRateXy as the amount of nanoErgs per 1mg of gold required for dexy.
- L41 defines lpRateXyIn as the ratio of reservesXIn divided by reservesYIn, representing the dexy price in the liquidity pool.
- L43 defines validOracleBox checking if the oracle data input box contains the correct oracleNFT.
- L45 defines validRateForRedeemingLp checking that validOracleBox is true and that the lpRateXyIn is greater than 0.98 times the oracleRateXy. So, checking that the LP price is at least 0.98 times the oracle price.
 - In the whitepaper r is defined as $r = \text{oracle price} / \text{lp price}$. So, the protocol defined in the paper checks that the lp price does not increase too much, since withdrawals should be stopped if $r \leq 0.98$. However, in the contract this condition reflects that $r \leq 1.02$, which means redeeming from the LP is valid as long as the LP price does not decrease too much. Is this intended behaviour?
 - So, in whitepaper, losing peg $\Rightarrow$ lp price going too high, while in contracts losing peg $\Rightarrow$ lp price going too low. Is this intended behaviour?
- L59-L65 defines validRedemption checking that:
 - The change in circulating supply of LP tokens is less than 0, meaning that LP tokens were added back into the liquidity pool and LP providers redeemed their ERG back.
 - The change in reserves of ERG in the liquidity pool is less than 0, meaning that ERG was withdrawn from the pool.
 - The change in reserves of dexy is less than 0, meaning that dexy was withdrawn from the liquidity pool. When a liquidity provider returns their LP tokens back into the pool they get back erg and dexy since they had to provide both pairs initially.
 - Checks that there is at least a 2% fee with the amount of erg withdrawn and the amount of dexy withdrawn.
 - That validRateForRdeemingLp is true.
- L67 defines selfPreserved checking that the LP Redeem box is recreated properly.

Transaction Diagram



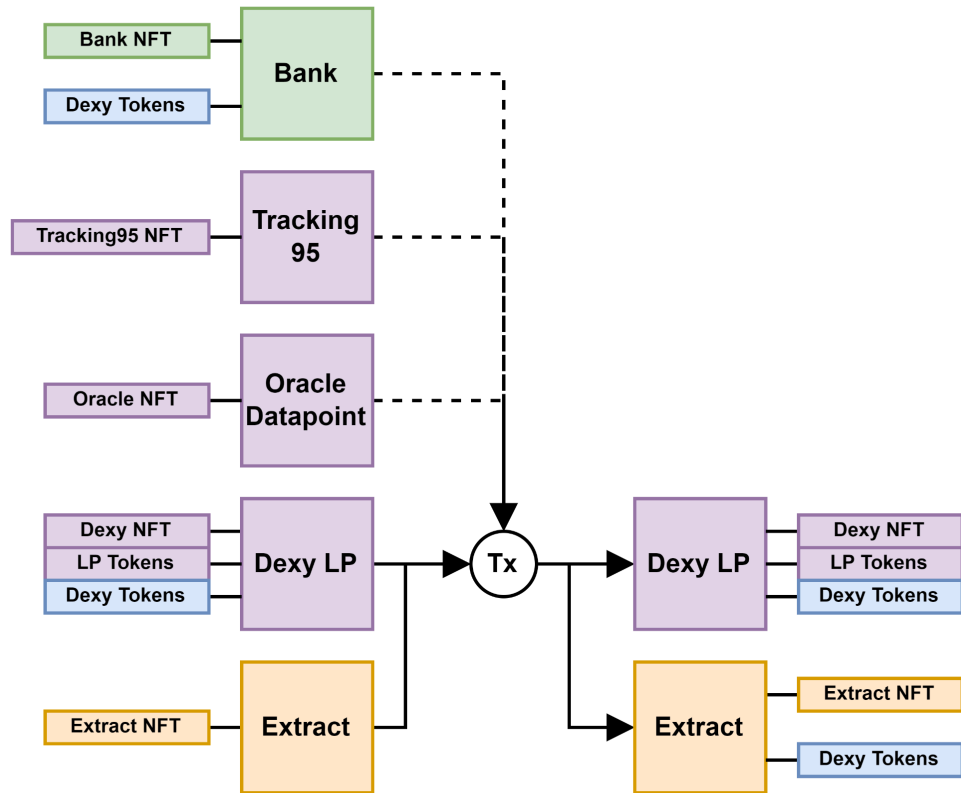
LP Extract for the Future Contract

- The contract extracts dexy coins from the liquidity pool if the price has depegged (i.e. gone lower with respect to the oracle price too much) and if the bank has no reserves, then returns the tokens to the liquidity pool once the peg in the LP has returned to within normal bounds.
- Contains the extractionNFT singleton identifier token and dexy tokens from after the extraction.
- L48 defines minBankNanoErgs as an amount equal to 10000 erg, so if the bank contains less than or equal to this amount then it will be considered in the “empty” state.
- L50-L54 defines the singleton NFTs as compile time constants for the different boxes referenced in the extract and release transactions.
- L56 defines T_extract as 720 blocks for which the rate r in the pool is below 0.95, so in order to extract, the liquidity pool must be depegged with $r \leq 0.95$ for at least 720 blocks.
- L57 defines T_release as 2 blocks for which the rate r in the pool is above 1.01, so in order to release the extracted funds back into the liquidity pool, the price must have increased above the oracle peg such that $r \geq 1.01$ for at least 2 blocks.
- L58 defines T_delay as 360 blocks, which represents the time between the extraction and release of funds back into the liquidity pool.
- L60 defines buffer as 5 blocks, which is an allowable error in blockheight.
- L62 defines updateNFT as a compile time constant
- L63 defines validUpdate checking that the first input box contains the updateNFT tokens as its first token.
 - This refers to updating the contract itself if needed in the future.
- L65 defines validAction checking if validUpdate is true otherwise continuing with the logic defining the extract action.
- L70 defines tracking95Box, which will check that the liquidity pool has $r \leq 0.95$ for T_extract amount of time.
- L71 defines tracking101Box, which will check that the liquidity pool has $r \geq 1.01$
- L82 defines lastBurnOrRelease as the creation height of the current extract contract box.
- L84 defines validDelay checking that T_delay amount of time has elapsed since the lastBurnOrRelease.
- L85 defines validSuccessor checking that the output extract box is recreated properly, especially that the output creation height is within the buffer error.
- L92 defines deltaDexy as the difference in amount of dexy between the output extract box and the input extract box, this value can be positive or negative.
- L94 defines validBankBox checking that if there are more than 2 data inputs then the following must true otherwise it is false:
 - The bank box is included by checking for its singleton token.
 - The bank box is empty according to the definition that it includes less than or equal to minBankNanoErgs amount of ergs.
- L99 defines validOracleBox checking that the oracle data input contains the correct oracleNFT singleton token.

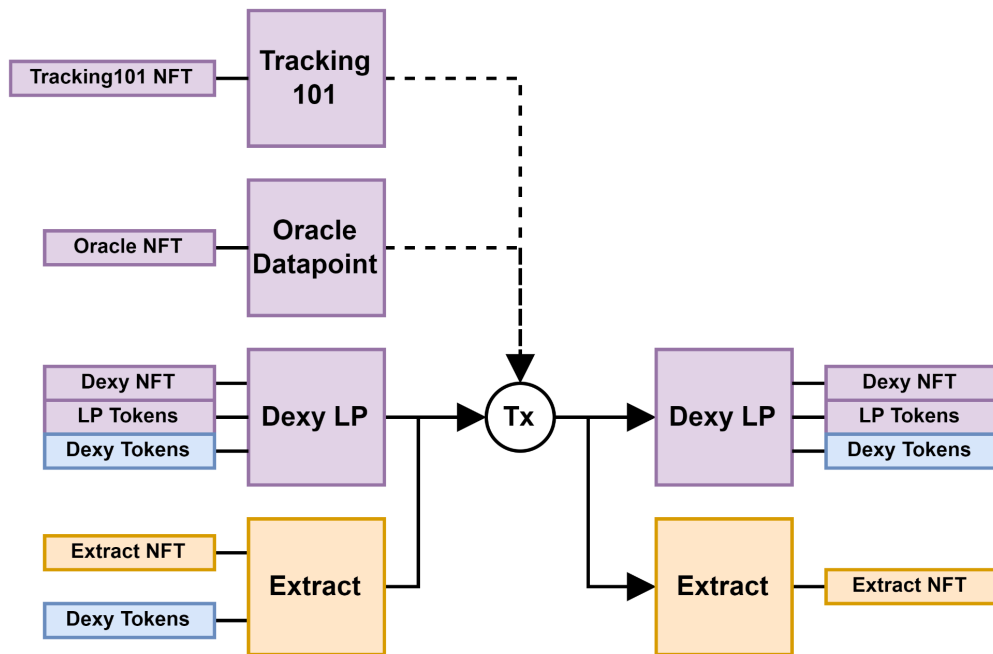
4EYES Labs

- L101-L104 defines the quantities for the assets in the liquidity pool input and output boxes.
- L106 defines validLpBox checking that the input LP box is correct, that the output LP box is recreated correctly and contains the correct amount of erg, lp tokens, and dexy tokens. It checks that the amount of dexy in the output LP box is equal to the amount in the input minus the amount added/removed from the extract contract.
- L115-L116 checks that the tracking data inputs contain their respective tracking NFT singleton tokens.
- L124-L125 defines validExtractAmount checking that the $r < 1.03$ and that $r > 1.02$ in the output liquidity pool.
 - The comments for these two lines contradict what is written in the code.
 - This code is meant to reflect the condition in the whitepaper that the bank should restore the r value in the LP box such that it goes from $r \leq 0.95$ to $r = 0.98$, which is the state of stopped withdrawals, but inequalities are reversed due to definition of r .
 - There seems to be a contradiction with how r is defined in the whitepaper and in the code and what is meant by the LP depegging from the oracle price.
 - Does depegging mean the LP price going below the oracle price too much or going above the oracle price too much?
 - Is $r = \text{oracle price} / \text{lp price}$ OR $\text{lp price} / \text{oracle price}$?
- L127 defines validReleaseAmount checking that $r > 0.99$ in the output LP box.
 - The comment for this line contradicts what is written in the code.
- L129 defines validExtract checking that:
 - deltaDexy is greater than 0, since the output extract box should have taken dexy out from the LP box and increased the value from the input extract box.
 - validTracking95Box is true such that $r \leq 0.95$ for T_{extract} amount of time at least.
 - That the current block height minus the height tracked inside tracker95Height is greater than T_{extract} .
 - validBankBox is true, so there is a valid bank box data input.
 - validExtractAmount is true, so the correct amount of dexy is being extracted from the liquidity pool.
- L135 defines validRelease checking that:
 - deltaDexy is less than 0, since the output extract box should have returned dexy back into the LP BOx and decreased in value from the input extract box.
 - validTracking101Box is true such that $r \geq 1.01$ for T_{release} amount of time at least.
 - That the current block height minus the height tracked inside tracker101Height is greater than T_{release} .
 - validReleaseAmount is true, so the correct amount of dexy is being returned to the liquidity pool.

Transaction Diagram: Extract to Future



Transaction Diagram: Reverse Extract to Future



LP Mint Contract

- Contract for depositing dexy/erg pair and getting LP tokens back.
- Contains singleton token identifier.
- L27 defines supplyLpIn as the amount of initial lp tokens, \$initialLp, which is a compile time constant, minus the amount of lp tokens in the liquidity pool.
- L37-L39 defines deltaSupplyLp, deltaReservesX, and deltaReservesY, as the change in circulating supply from the output lp box and the input lp box, the change in erg reserves from the output and input lp box, and the change in dexy reserves from the output and input lp box, respectively.
- L42 defines validMintLp checking that:
 - The change in circulating supply of LP tokens is positive, thus the output liquidity pool contains fewer LP tokens.
 - The change in erg reserves is positive, thus erg was added into the output liquidity pool.
 - The change in dexy reserves is positive, thus dexy was added into the output liquidity pool.
 - The change in circulating supply of LP tokens is less than or equal to amount called sharesUnlocked defined as the minimum of the two following quantities:
 - The change in erg reserves divided by the erg reserves inside the input liquidity pool, times the input circulating supply of LP tokens.
 - The change in dexy reserves divided by the dexy reserves inside the input liquidity pool, times the input circulating supply of LP tokens.
 - Thus, the amount of LP tokens “minted” is proportional to the amount of erg or dexy added into the liquidity pool relative to their respective reserve amounts, i.e. proportional to the increase in reserves. This maintains a balance between the LP token supply and the total reserves in the pool.
 - By taking the minimum, it prevents dilution of existing LP token holders.
- L50 defines selfPreserved checking that the output lp mint box has the same contract, an amount of erg greater or equal to the input lp mint box, and that the tokens are preserved.

LP Swap Contract

- Contains singleton token NFT identifier.
- L39-L40 defines the fee numerator and denominator.
- L51-L58 defines the reserve amounts for lp tokens, erg, and dexy in the input and output liquidity pool box.
- L68 defines deltaSupplyLp as the change in circulating supply of LP tokens after the transaction minus the circulating supply before the transaction.
- L69 defines deltaReservesX as the change in erg amount of the output LP box minus the input LP box.
- L70 defines deltaReservesY as the change in dexy amount of the output LP box minus the input LP box.
- L72 defines validSwap checking that:
 - deltaSupplyLp is equal to 0, thus no LP tokens should be removed from the liquidity pool in a swap transaction.
 - If deltaReservesX is greater than 0, then make sure the correct amount of dexy tokens are removed from the liquidity pool following the inequality on L75.
 - Otherwise, dexy is being added to the liquidity pool and erg is being removed, which will have a slightly modified inequality.

- These two equations do not follow the derivation outlined in the comments in L20-L25. It should instead be the following:

```
// If deltaReservesX > 0 => erg being added, dexy being removed.
deltaReservesX * feeNum * reservesYIn.toBigInt <= (reservesXIn.toBigInt *
deltaReservesY + deltaReservesX * reservesYIn.toBigInt) * feeDenom

// Otherwise, deltaReservesX < 0 => erg being removed, dexy being added.
deltaReservesY * feeNum * reservesXIn.toBigInt <= (reservesYIn.toBigInt *
deltaReservesX + deltaReservesY * reservesXIn.toBigInt) * feeDenom
```

- L80 defines selfPreserved checking that the output lp swap contract is recreated properly:
 - The contract remains the same.
 - The output contains greater than or equal amounts of erg than the input box.
 - The tokens remain the same.

Tracking Contract

- Contains the TrackingNFT singleton token identifier.
- R4 is the oracle / LP rate ratio numerator as an Int type.
- R5 is the oracle / LP rate ratio denominator as an Int type.
- R6 is the isBelow Boolean flag indicating what type of tracking is being done, either above or below the oracle / LP rate ratio.
- R7 contains the trackingHeight blockheight when the tracker is set in the “triggered” state.
 - This will be Int.MaxValue when the tracker is in the “reset” state.
- The tracking box is used in one transaction for updating the tracker state.
 - The tracking box is the only input and output.
 - The oracle datapoint and the LP box are both data-inputs.
- L36 defines threshold as the error in the trigger block height, the value is set to 3.
- L49 defines tokenY as the dexy tokens in the LP data-input box.
- L51 defines validLp checking that the LP data-input box contains the correct lpNFT singleton identifier token.
- L53 defines validOracleBox checking that the oracle data-input box contains the correct oracleNFT singleton identifier token.
- L54 defines validSuccessor checking that:
 - The tokens in the input and output tracking boxes remain the same.
 - The tracking contract remains the same.
 - The output tracking box contains greater than or equal to the amount of ERG in the input tracking box.
- L61 defines oracleRateXY as the oracle rate as the nanoErg per 1mg of gold.
- L62 defines reservesX as the amount of ERG in the LP data-input box.
- L63 defines reservesY as the amount of dexy tokens in the LP data-input box.
- L64 defines lpRateXY as the amount of ERG in the LP data-input box divided by the amount of dexy tokens in the LP data-input box.
- L78 comment says that the event will be triggered if $r \leq 0.98$ for arbitrage mint, however, in the arbitrage mint contract it is set for $r \geq 1.01$
- L93-L106 comments contradict each other, it states that $r = \text{oracle rate} / \text{LP rate}$, but the checking conditions in the comments are defined such that $r = \text{LP rate} / \text{oracle rate}$.
- L108 defines x as the LP rate multiplied by the denominator in the input tracking box.
- L109 defines y as the numerator of the input tracking box multiplied by the oracle rate.
- L111 defines notTriggeredEarlier checking that the trackingHeightIn equals \$intMax compile time constant representing Int.MaxValue as the infinity value.
- L112 defines triggeredNow checking that:
 - The tracking height in the output tracking box is greater than or equal to the current height minus the threshold.
 - The tracking height in the output tracking box is less than or equal to the current height.
- L115 defines notResetEarlier checking that the input tracking height is less than Int.MaxValue.

4EYES Labs

- L116 defines resetNow checking that the output tracking height equals Int.MaxValue
- L118 defines trigger checking that:
 - Either isBelowIn is true and that $x < y$ or isBelowIn is false and $x > y$.
 - And that notTriggeredEarlier is true.
 - And that triggeredNow is true.
- L119 defines reset checking that:
 - Either isBelowIn is true and $x \geq y$ or isBelowIn is false and $x \leq y$.
 - And that notResetEarlier is true.
 - And that resetNow is true.
- L120 defines correctAction checking that trigger or reset is true.
- L122-L125 checks that the input and output numerator, denominator, and isBelow status are preserved, and that a correctAction is true.

Deposit Proxy Contract

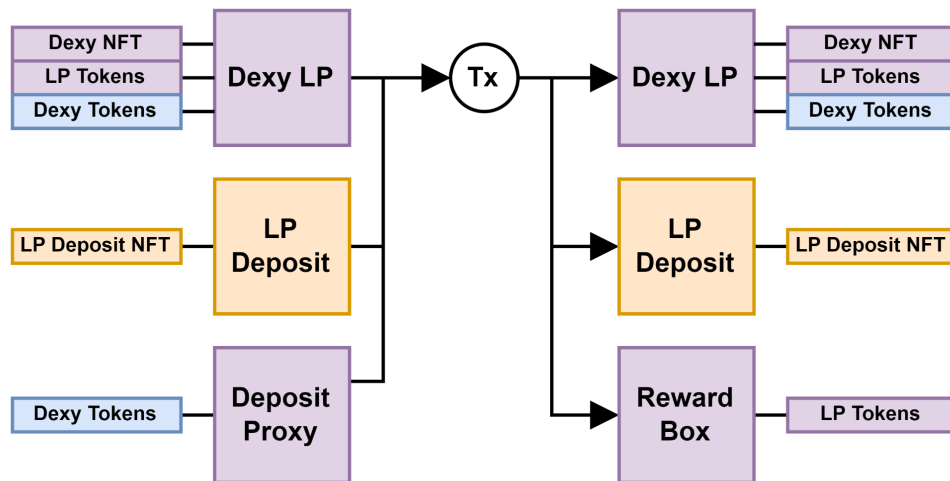
- The proxy contract to deposit erg/dexy liquidity pair to the liquidity pool.
- L9 defines InitiallyLockedLP as the hex value of 9223372036854775807 as a Long type.
- L13 defines validDeposit checking that if the amount of inputs is greater than or equal to 2 and the input liquidity pool contains exactly 3 tokens then:
 - L20 defines validPoolIn checking that the input LP contains the PoolNFT singleton identifier.
 - L27 defines supplyLP as the InitiallyLockedLP minus the amount of lp tokens in the input liquidity pool. This value represents the current circulating supply of LP tokens.
 - L29 defines _selfX as the value stored in a compile time constant called SelfX.
 - This represents the amount to deposit into the liquidity pool, which will be smaller than the amount of erg inside the deposit proxy contract to account for any fees.
 - L31 defines minByX as the multiple of _selfX and supplyLP divided by reservesXAmount. This represents the amount of LP tokens to give based on the added erg amount to the pool.
 - L32 defines minByY as the multiple of the amount of dexy tokens in the deposit proxy contract and supplyLP divided by reservesYAmount. This represents the amount of LP tokens to give based on the added dexy amount to the pool.
 - L34 defines minimalReward as the minimum of minByX and minByY.
 - L36 defines rewardLP as the amount of lp tokens given in the output user box.
 - L38 defines validChange checking that if minByX is less than minByY and the output reward box contains 2 tokens then:
 - L40 defines diff as the difference of minByY minus minByX.
 - L41 defines excessY as the multiple of diff and reservesYAmount divided by supplyLP. Since minByX is less than minByY, the amount of LP tokens will be determined by minByX, thus an excess amount of dexy tokens will be given back to the user, i.e. whatever is not put into the liquidity pool will be bounded above by an amount equal to excessY.
 - L43 defines changeY as the dexy tokens in the output reward box.
 - L45 checks that:
 - the output reward box erg value is greater than or equal to the current input deposit proxy amount minus the DexFee compile time constant minus _selfX.
 - The output dexy tokens in the reward box have the same token id as the dexy tokens inside the liquidity pool.
 - The amount of dexy tokens in the output reward box is greater than or equal to the amount defined by excessY.
 - Otherwise, minByX is greater than or equal to minByY where:
 - L50 defines diff as the difference of minByX minus minByY.
 - L51 defines excessX as the multiple of diff and reservesXAmount divided by supplyLP. Since minByX is greater than or equal to minByY, the

4EYES Labs

amount of LP tokens will be determined by minByY, thus an excess amount of erg will be given back to the user, i.e. whatever is not put into the liquidity pool will be bounded above by an amount equal to excessX.

- L53 checks that the output reward box erg value is greater than or equal to the current input deposit proxy amount minus the DexFee compile time constant minus the difference of _selfX and excessX.
- L56 otherwise, validChange is set to false.
- L59 defines validMinerFee checking that the sum of erg values in any output box that is a miner fee box, is less than or equal to an amount defined by the compile time constant MaxMinerFee.
- L63-L68 checks that:
 - A valid liquidity pool is used as an input.
 - The reward output box is owned by the address defined in the compile time constant Pk.
 - validChange evaluates to true.
 - The lp token id in the reward box matches the lp token id inside the liquidity pool.
 - The amount of lp tokens in the reward box is greater than or equal to the minimalReward amount.
 - validMinerFee evaluates to true.
- L70 otherwise, validDeposit is set to false.
- L72 also considers that the public key stored in the compile time constant Pk can spend the box as well, as a potential refund mechanism.

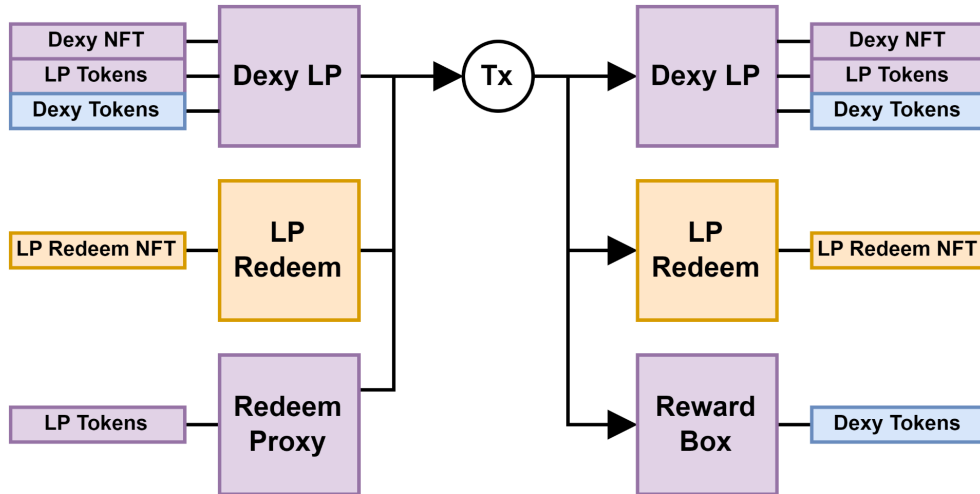
Transaction Diagram



Redeem Proxy Contract

- The proxy contract to redeem erg/dexy from the liquidity pool by returning LP tokens to the pool.
- L10 defines InitiallyLockedLP as the hex value of 9223372036854775807 as a Long type.
- L14 defines validRedeem check that if the amount of inputs is greater than or equal to 2 and the input liquidity pool contains 3 tokens then:
 - L21 defines validPoolIn checking that the input liquidity pool contains an NFT with token id equal to the compile time constant PoolNFT.
 - L27 defines supplyLP as the difference of InitiallyLockedLP minus the amount of LP tokens inside the input liquidity pool. This value corresponds to the circulating supply of LP tokens.
 - L29 defines minReturnX as the amount of erg returned determined by the amount of LP tokens owned as a fraction of the circulating supply multiplied by the amount of erg reserves in the liquidity pool, multiplied by 0.98 to impose a 2% withdrawal fee.
 - L30 defines minReturnY as the amount of dexy returned determined by the amount of LP tokens owned as a fraction of the circulating supply multiplied by the amount of dexy reserves in the liquidity pool, multiplied by 0.98 to impose a 2% withdrawal fee.
 - L32 defines returnXAmount as erg inside the return output box minus the amount of erg inside the redeem proxy contract plus the DexFee compile time constant amount. This value isolates the contribution due only to the erg removed from the liquidity pool.
 - L35-L37 defines validMinerFee checking that the sum of erg values in any output box that is a miner fee box, is less than or equal to an amount defined by the compile time constant MaxMinerFee.
 - L39-L44 checks that:
 - validPoolIn evaluates to true.
 - The return output box is owned by the address defined in the compile time constant Pk.
 - The return output box contains a token with the same token id as the dexy tokens inside the liquidity pool.
 - returnXAmount is greater than or equal to minReturnX.
 - The amount of dexy tokens in the return output box is greater than or equal to the minReturnY amount.
 - validMinerFee evaluates to true.
- L46 otherwise, validRedeem is set to false.
- L48 also considers that the public key stored in the compile time constant Pk can spend the box as well, as a potential refund mechanism.

Transaction Diagram



SwapBuy Proxy Contract

L4-L5 defines the fee denominator and numerator.

L6-L7 defines the dex fee per token numerator and denominator.

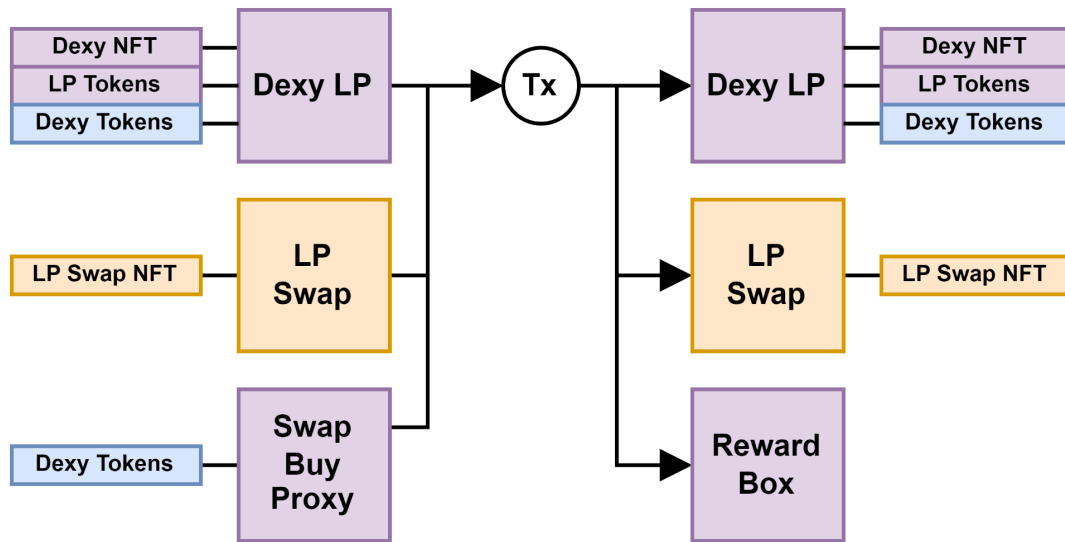
L8 defines the minimum quote amount.

L10 defines poolIn as the input LP box being the first input to the transaction.

L12 defines validTrade checking that:

- If the amount of inputs equals 3 and the LP box contains 3 different tokens then:
 - L15 defines rewardBox as the third output box.
 - L17 defines baseAmount as the amount of dexy tokens in the current swap buy proxy contract.
 - L19 defines poolNFT as the singleton identifier of the Dexy liquidity pool.
 - L21 defines poolReservesX as the amount ERG inside the Dexy LP box.
 - L22 defines poolReservesY as the amount of Dexy tokens inside the Dexy LP box.
 - L24 defines validPoolIn checking that the poolNFT inside the dexy LP pool box matches the PoolNFT compile time constants.
 - L26 defines deltaNErgs as the difference between the amount of ERG in the reward box and the current swap buy proxy contract box.
 - L27 defines quoteAmount as the deltaNErgs divided by the one minus the dex fee per token. Thus, quoteAmount will have units of dexy tokens, representing an upper bound for how many dexy tokens should have been added into the pool.
 - L28 defines relaxedOutput as quoteAmount plus one to account for integer division errors.
 - L29 defines fairPrice, which checks the value of an inequality derived from the AMM formula requirements.
 - Not clear how this derivation is obtained from the AMM formulas.
 - L31-L33 defines validMinerFee checking that any output box matching the miner fee contract has cumulative value less than the MaxMinerFee compile time constant.
 - L35-L39 checks that:
 - A valid LP pool is an input.
 - The rewardBox contract bytes match the redeemer contract bytes.
 - That the quoteAmount is greater than or equal to the MinQuoteAmount compile time constant.
 - fairPrice is true.
 - validMinerFee is true.
- Otherwise, false.
- L42 checks that a valid refund is initiated or a valid trade.

Transaction Diagram



SwapSell Proxy Contract

L4-L5 defines the fee denominator and numerator.

L6-L7 defines the dex fee per token numerator and denominator.

L8 defines the minimum quote amount.

L9 defines the base amount required for the swap.

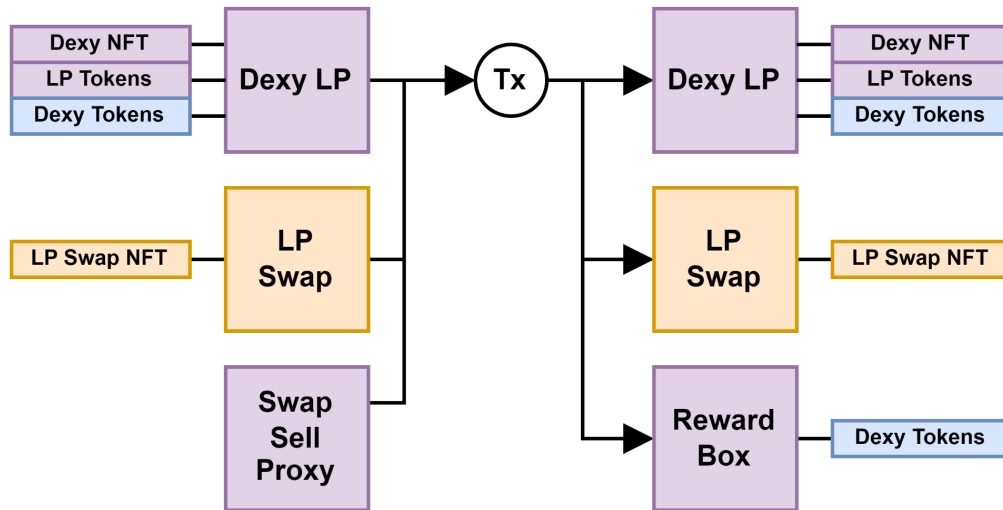
L11 defines poolIn as the input LP box being the first input to the transaction.

L13 defines validTrade checking that:

- If the amount of inputs equals 3 and the LP box contains 3 different tokens then:
 - L16 defines rewardBox as the third output box.
 - L18 defines poolNFT as the singleton identifier of the Dexy liquidity pool.
 - L20 defines poolY as the dexy tokens inside the LP pool.
 - L22 defines poolReservesX as the amount ERG inside the Dexy LP box.
 - L23 defines poolReservesY as the amount of Dexy tokens inside the Dexy LP box.
 - L24 defines validPoolIn checking that the poolNFT inside the dexy LP pool box matches the PoolNFT compile time constants.
 - L26 defines quoteAsset as the first token in the output rewardBox, which will be the dexy tokens emitted from the LP pool box.
 - L27 defines quoteAmount as the amount of dexy tokens in the output reward box.
 - L29 defines fairDexFee checking that the output rewardBox value is greater than or equal to the amount of ERG in the current swap sell proxy contract input box minus the quoteAmount multiplied by the dex fee per token, minus the BaseAmount compile time constants. This basically checks that the user has enough ERG to pay the dex fee.
 - L31 defines relaxedOutput as quoteAmount plus one to account for integer division errors.
 - This does not seem to be needed since there is no rounding loss to account for.
 - L32 defines fairPrice, which checks the value of an inequality derived from the AMM formula requirements.
 - Not clear how this derivation is obtained from the AMM formulas.
 - L34-L36 defines validMinerFee checking that any output box matching the miner fee contract has cumulative value less than the MaxMinerFee compile time constant.
 - L38-L44 checks that:
 - A valid LP pool is an input.
 - The rewardBox contract bytes match the redeemer contract bytes.
 - The quoteAsset token id matches the Quoteld compile time constant.
 - That the quoteAmount is greater than or equal to the MinQuoteAmount compile time constant.
 - fairDexFee is true.
 - fairPrice is true.
 - validMinerFee is true.

- Otherwise, false.
- L47 checks that a valid refund is initiated or a valid trade.

Transaction Diagram



Appendix

Issue Guide

Severity	Description
CRITICAL	The contract contains one or more serious errors and warnings.
SERIOUS	An error that will cause unexpected behavior.
WARNING	A property that is questionable but will not cause unexpected behavior.
GOOD	Everything works as expected.

Status	Description
RESOLVED	The issues have been addressed.
UNRESOLVED	The issues have not been addressed.

About Us

4EYES is an ErgoScript design and auditing firm for Ergo blockchain.

Linktree: <https://linktr.ee/4EYESLabs>